

Interrupciones externas en el ESP32

Sistemas Digitales · Unidad 1: Microcontroladores para sistemas embebidos

Semana 4 · Miércoles 4 de noviembre de 2026

Carrera de Computación · Universidad Nacional de Loja · Docente: Ing. Luis Sinche Cueva, Mg.

OBJETIVO

Objetivo de la sesión

Explicar cómo el ESP32 detecta eventos externos en un pin mediante interrupciones, y escribir una ISR correcta (corta, en IRAM, con variables volatile y patrón bandera) para que ningún evento se pierda.

Resultado R1

Diseña arquitectura de microcontroladores.

Se usa en

AP4 del viernes: sistema de alerta doméstica.

Al final

Control de aprendizaje: 5 preguntas, 10 minutos.

MAPA

Ruta de hoy

1. Sondeo frente a interrupción
2. Flancos
3. Pines que sirven
4. attachInterrupt()
5. **La ISR a fondo**
6. **IRAM: dónde vive la ISR**
7. **volatile**
8. **Patrón bandera**
9. Rebote dentro de la ISR
10. Todas las piezas juntas

En amarillo: el corazón de la clase.

ACTIVACIÓN

¿Qué hicimos en las semanas 2 y 3?

AP2 · Juego de luces

El loop() preguntaba en cada vuelta:
"¿ya presionaron el botón?"

→ Hoy le pondremos nombre: sondeo.

AP3 · Temporizador

El temporizador avisaba solo, sin que el loop() preguntara.

→ Eso era una interrupción.

Hoy: un botón (como la AP2) que avisa con interrupción (como la AP3).

GLOSARIO 1/2

Palabras que usaremos hoy

Término	Qué significa
Sondeo (polling)	Preguntar una y otra vez, en el loop(), si algo ya pasó.
Interrupción	Aviso del hardware que pausa el programa para atender algo urgente.
Interrupción externa	Interrupción cuyo aviso llega desde fuera del chip, por un pin.
ISR	Interrupt Service Routine: la función que se ejecuta al llegar el aviso.
Flanco	El instante en que una señal cambia de 0 a 1 o de 1 a 0.
Rebote	Varios cambios falsos y rapidísimos que produce un botón mecánico.

GLOSARIO 2/2

Más palabras (las del bloque central)

Término	Qué significa
Compilador	Programa que traduce tu C a instrucciones del chip. Intenta ahorrar pasos.
Registro de la CPU	Casillas muy rápidas dentro de la CPU donde trabaja con copias de datos.
Flash	Memoria grande y lenta, fuera del núcleo, donde se guarda el programa.
SRAM / IRAM	Memoria rápida dentro del chip. IRAM es la parte para instrucciones.
Caché	Copia pequeña y rápida de lo que la CPU leyó hace poco de la flash.
Watchdog	"Perro guardián": reinicia el chip si algo se queda trabado mucho tiempo.

Sondeo: mirar por la ventana

La analogía

Esperas un paquete. Cada 5 minutos te levantas a mirar por la ventana: "¿ya llegó?"

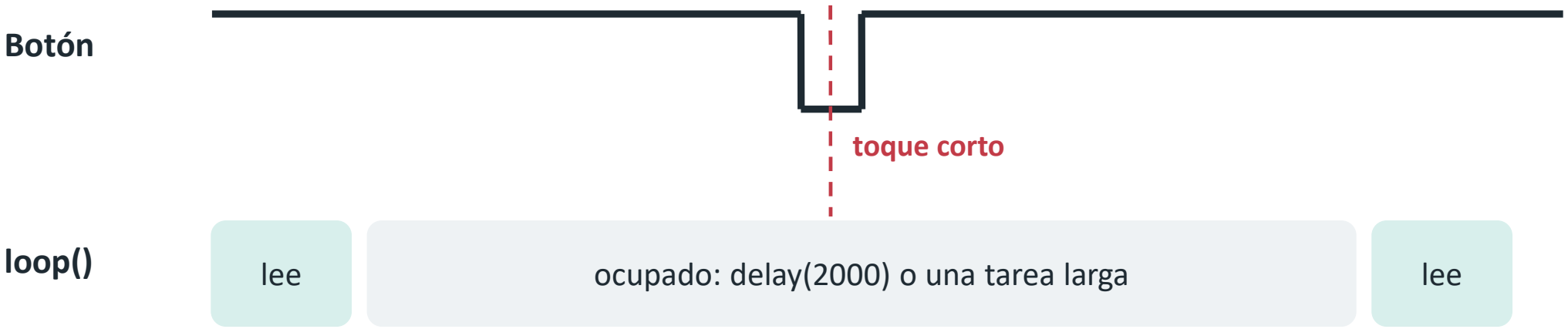
En el ESP32

El loop() lee el pin en cada vuelta.

```
void loop() {
  if (digitalRead(21) == LOW) {
    // ¿ya presionaron?
    encenderAlarma();
  }
  // ...otras tareas...
}
```

BLOQUE 1 · SONDEO

El problema: si el loop está ocupado, el toque se pierde



Resultado: cuando el loop() vuelve a leer, el botón ya está suelto. El evento **se perdió**. En un juego de luces da igual; en una alarma de puerta, no.

Interrupción: instalar un timbre

1

Suena el timbre

Interrupción externa: llega un aviso por un pin.

2

Vas a abrir

ISR: la función que atiende el aviso.

3

Vuelves a lo tuyo

El programa sigue justo donde estaba.

Nadie mira por la ventana: el aviso llega solo.

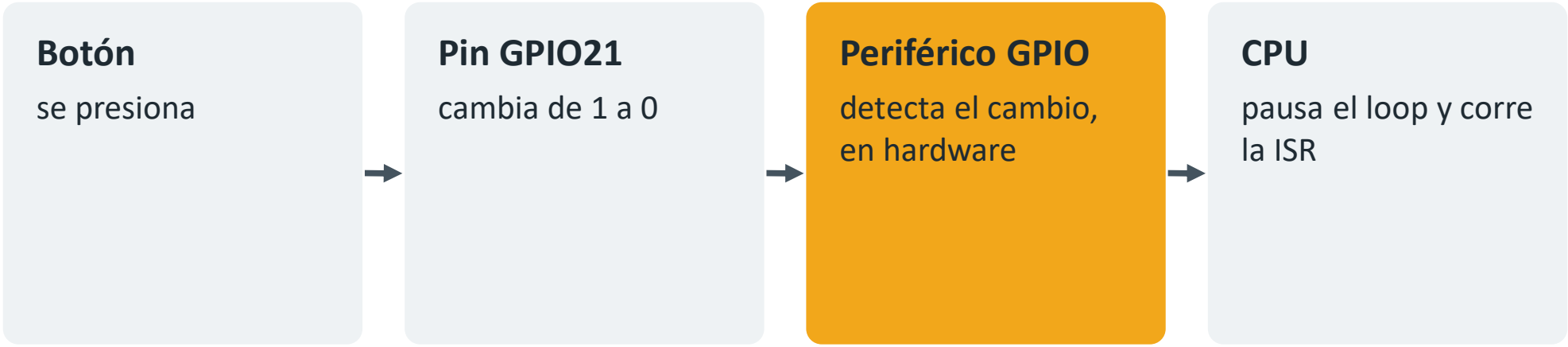
Qué hace el hardware cuando llega el aviso



1. **Pausa** el loop() en la instrucción en que iba.
2. **Ejecuta** la ISR completa.
3. **Regresa** a la instrucción siguiente, como si nada.

Mientras corre la ISR, el loop() está congelado.

¿Quién vigila el pin? El periférico GPIO, no tu código



El que mira es un circuito, no el programa. Por eso el aviso llega aunque el loop() esté ocupado en un delay(). Periférico = bloque de hardware del chip que trabaja por su cuenta.

Semana 3 frente a semana 4: el mismo mecanismo

	Semana 3: temporizador	Semana 4: externa
¿Quién toca el timbre?	El temporizador, dentro del chip	Algo de afuera, por un pin
¿Cuándo?	Cada cierto tiempo, fijo	Cuando ocurre el evento
Se configura con	timerBegin, timerAlarm…	attachInterrupt()
Pausa, ISR, regreso	Igual	Igual
ISR, IRAM_ATTR, volatile, bandera	Igual	Igual

Nivel frente a flanco

Nivel

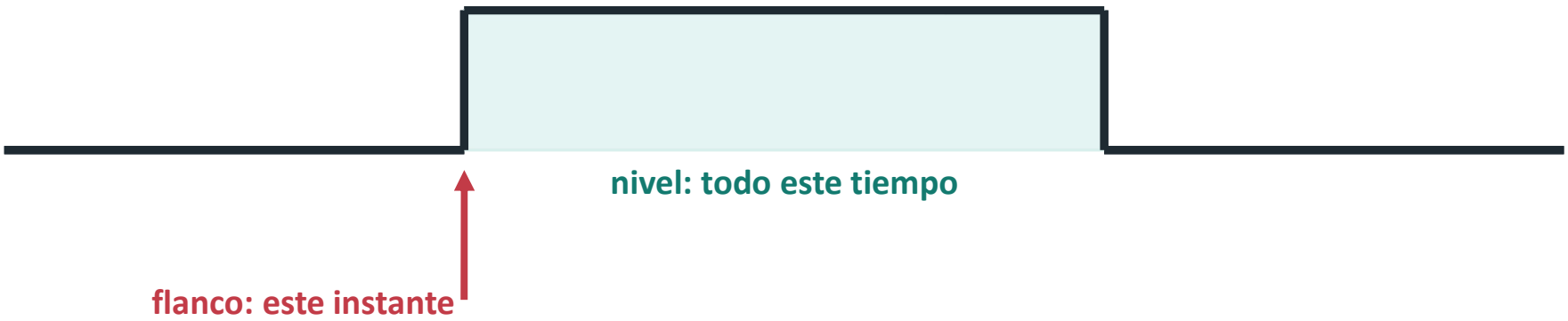
"La puerta está abierta."

Un estado que dura.

Flanco

"La puerta se acaba de abrir."

Un instante.



BLOQUE 2 · FLANCOS

Recordatorio: con INPUT_PULLUP la lógica es invertida

Suelto

La pull-up interna jala el pin hacia 3,3 V.

1

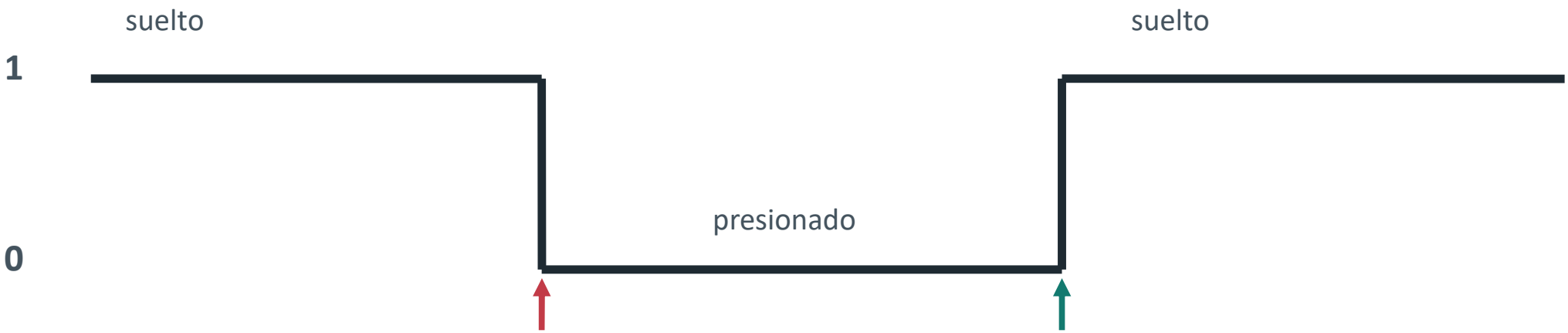
Presionado

El botón une el pin con GND.

0

Suelto = 1 · Presionado = 0 (al revés de lo que uno espera)

Los tres modos: RISING, FALLING y CHANGE



FALLING (1→0)

Ocurre al presionar.

RISING (0→1)

Ocurre al soltar.

CHANGE

Los dos: la ISR corre dos veces por toque.

BLOQUE 2 · ACTIVIDAD

¿Qué modo usa la alarma?

El botón está en INPUT_PULLUP. La alarma debe sonar en el instante en que alguien presiona. ¿RISING, FALLING o CHANGE? ¿Por qué?

Truco del ascensor

RISING: la señal sube.

FALLING: la señal cae.

Pista

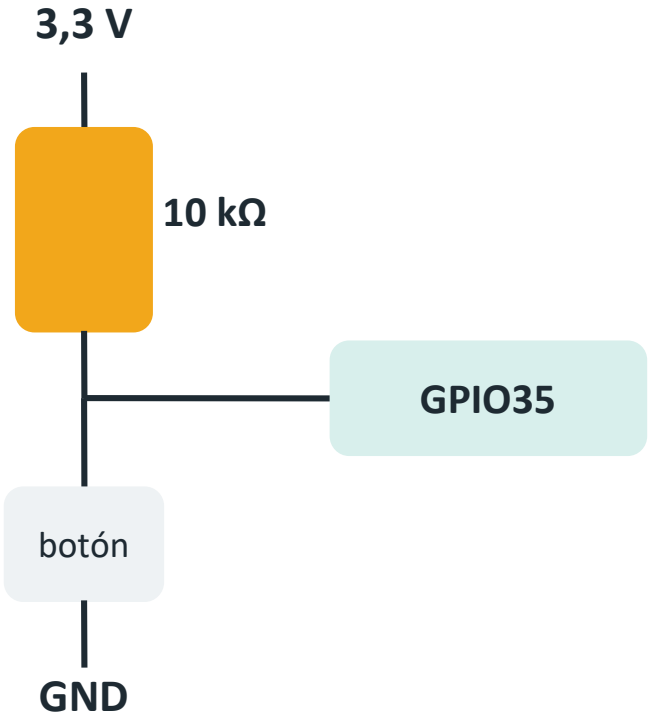
Con pull-up, ¿presionar hace subir o caer la señal?

¿Qué pines del ESP32 admiten interrupción?

Pines	¿Sirven?	Cuidado
16, 17, 18, 19, 21, 22, 23, 25, 26, 27, 32, 33	**Sí**	Los recomendados.
34, 35, 36, 39 (solo entrada)	**Sí, con cuidado**	No tienen pull-up interna.
0, 2, 5, 12, 15 (strapping)	**Evitar**	Si están presionados al encender, el chip arranca mal.
6 a 11 (flash)	**No**	Los usa la memoria del programa.

En el Arduino Uno solo D2 y D3 podían. En el ESP32, casi todos. AP4: botón en GPIO21.

Pines 34–39: hay que poner la pull-up por fuera



Sin la resistencia

El pin queda flotando: capta ruido y aparecen alarmas sin que nadie toque.

Con la resistencia

Suelto = 1 seguro. Hace por fuera lo que la pull-up interna hace por dentro.

attachInterrupt(): la instalación del timbre

```
attachInterrupt(digitalPinToInterrupt(21), isrBoton, FALLING);
```

1 ¿Qué pin vigilar?

digitalPinToInterrupt(21)

2 ¿A quién llamar?

isrBoton (sin paréntesis: le das el nombre, no la llamas)

3 ¿Qué cambio?

FALLING, RISING o CHANGE

En español: "cuando el pin 21 caiga de 1 a 0, llama a isrBoton".

Dónde va: una sola vez, en setup()

```
void setup() {
  pinMode(21, INPUT_PULLUP);  // 1.°
  attachInterrupt(
    digitalPinToInterrupt(21),
    isrBoton, FALLING);       // 2.°
}
```

Una sola vez

El timbre se instala una vez; no cada vez que llega alguien.

Orden

Primero pinMode, después attachInterrupt.

detachInterrupt(pin)

Desinstala el timbre.

¿Qué es una ISR?

Función normal

Tú la llamas en tu código, en el punto que decides.

```
saludar();
```

ISR

Tú nunca la llamas. La llama el hardware cuando llega el aviso, en cualquier momento.

ISR = Interrupt Service Routine
Rutina de servicio de interrupción: la función que atiende el aviso. En la semana 3 la llamamos callback.

Analogía: dejar tu número en el taller

1

Dejas tu número

`attachInterrupt(…, isrBoton, …)`

2

Sigues con tu vida

El loop() hace lo suyo.

3

El taller te llama

El hardware ejecuta isrBoton.

Llamar al taller cada hora "¿ya está?" sería sondeo.

Regla 1: la ISR debe ser corta

✓ Sí va en la ISR

- Subir una bandera
- Guardar el tiempo: millis()
- Sumar 1 a un contador
- digitalWrite (es rápido)

✗ No va en la ISR

- Serial.print / println
- delay()
- Cálculos largos
- Esperar a que algo pase

Como el extintor: apagas el fuego y sales. Nada de limpiar la cocina.

Si la ISR tarda demasiado: el watchdog

ISR larga
Serial, delay…

→

Watchdog
detecta que algo está trabado

→

Reinicio
el ESP32 arranca de nuevo

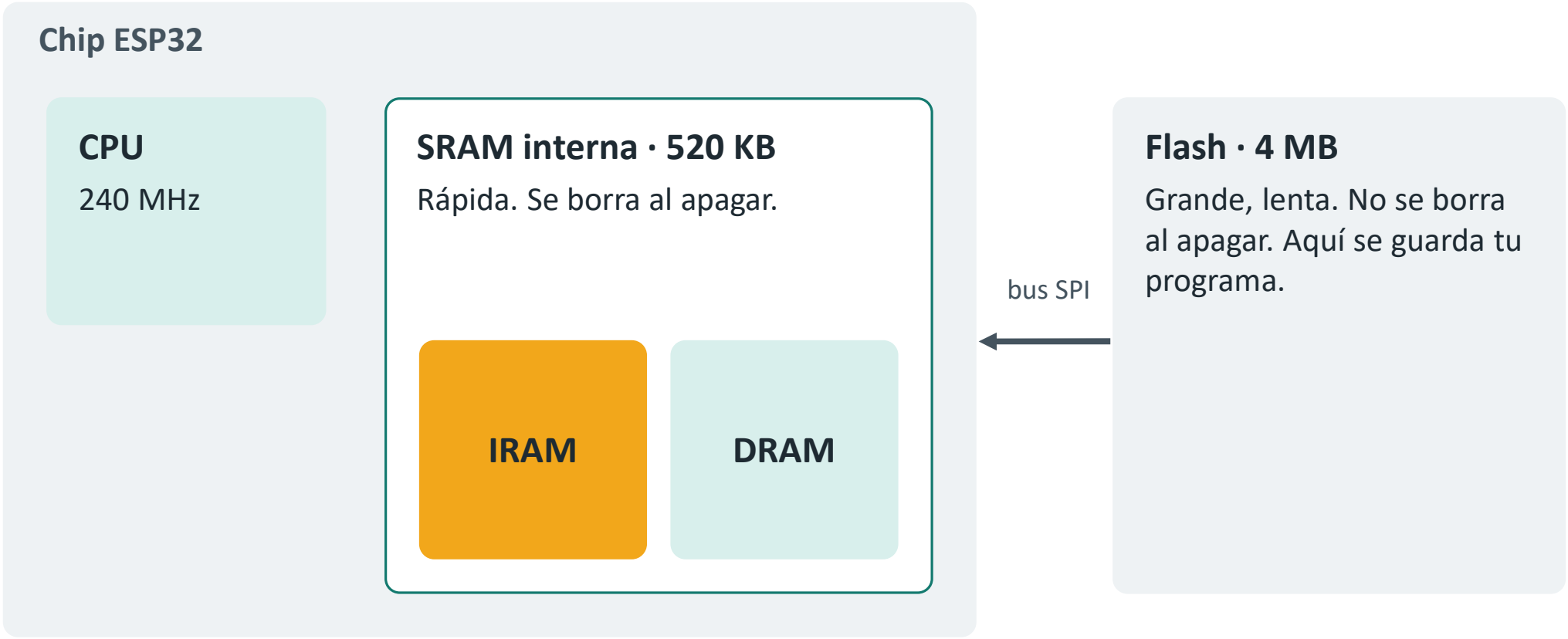
→

Monitor Serie
"Guru Meditation Error"

Analogía

El guardia de la fábrica: si ve una máquina trabada demasiado tiempo, la apaga y la vuelve a encender. El síntoma típico: "el ESP32 se reinicia de vez en cuando".

Dónde vive el programa: flash y SRAM



BLOQUE 6 · IRAM

IRAM y DRAM: dos zonas de la misma SRAM

IRAM · Instruction RAM

Guarda instrucciones: código que la CPU ejecuta.

Aquí pondremos la ISR.

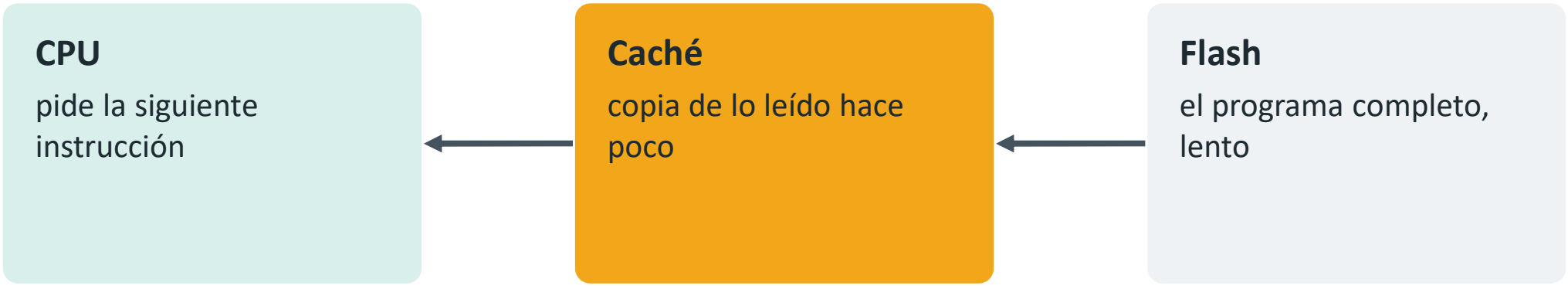
DRAM · Data RAM

Guarda datos: por ejemplo, las variables globales.

Aquí vive la variable alerta.

Ojo: en el ESP32, DRAM significa "Data RAM", no la DRAM de las memorias de un computador.

La caché: cómo la CPU lee rápido de una flash lenta



Analogía

La biblioteca (flash) es enorme pero queda lejos. En tu mesa (caché) tienes los libros que usaste hace poco: casi siempre lo que necesitas ya está ahí.

BLOQUE 6 · IRAM

El problema: a veces la caché se apaga

1

El ESP32 escribe en la flash

por ejemplo, al guardar datos.

→

2

La caché se apaga

un momento.

→

3

Llega una interrupción

y la ISR está en la flash.

→

4

La CPU no puede leerla

el chip se cae.

Es raro, pero real. Y cuando pasa, casi no se puede reproducir: "a veces falla y no sé por qué".

Analogía: el extintor a la mano

✗ En la bodega (flash)

A veces la bodega está cerrada (caché apagada). Si hay fuego justo entonces, no alcanzas el extintor.

✓ En la pared de la cocina (IRAM)

Siempre a la mano, pase lo que pase.

La ISR es el código de emergencia: tiene que estar siempre a la mano.

IRAM_ATTR: "guarda esta función en la IRAM"

```
void IRAM_ATTR isrBoton() {
  // código corto de emergencia
}
```



entre el tipo y el nombre

Si la omites

Funciona casi siempre y falla muy de vez en cuando.

Regla

Toda ISR lleva IRAM_ATTR. Siempre.

Dos lugares donde puede estar un dato

RAM → el pizarrón

Donde vive la variable alerta. La ISR y el loop() la pueden ver.

Registro de la CPU → tu cuaderno

Casillas dentro de la CPU. Para trabajar con un dato, la CPU primero lo copia aquí.

No son los registros de periféricos de la semana 2 (GPIO_OUT, GPIO_IN).

El compilador es "ahorrativo"

```
while (alerta == false) {
  // nada aquí adentro
  // cambia alerta
}
```

El compilador piensa:
"Nadie cambia alerta aquí. La copio una vez a mi cuaderno y no vuelvo al pizarrón."

El problema: el compilador no sabe que existe la ISR. Nadie la llama desde el código: la dispara el hardware. Por eso cree que nadie cambia alerta.

Analogía: la fecha del examen

1

Copias del pizarrón

El pizarrón dice "viernes" y lo copias a tu cuaderno.

2

El profesor lo cambia

Pone "lunes". El profesor es la ISR.

3

Miras tu cuaderno

Sigue diciendo "viernes". Nunca te enteras.

Para no perderte el cambio: mira el pizarrón cada vez.

Lo mismo, dentro del chip

Momento	RAM (pizarrón)	Copia en la CPU (cuaderno)	¿Qué ve el loop?
Arranca el loop	alerta = false	false	false
La ISR escribe	**alerta = true**	**false**	**false**
Siguen las vueltas	true	**false**	**false para siempre**

El LED nunca se enciende, y el código "se ve bien": no hay ningún error de compilación.

Lo que hace volatile

```
volatile bool alerta = false;
```

"Esta variable puede cambiar a tus espaldas. No confíes en tu copia: ve a la RAM cada vez que la uses."

En la analogía

Mira siempre el pizarrón.

Solo hace eso

No la hace más rápida ni más segura.

BLOQUE 7 · ACTIVIDAD

La prueba: ¿la usan la ISR Y el loop?

Variable	¿La toca la ISR?	¿La toca el loop?	¿volatile?
alerta	escribe	lee	**Sí**
pulsos	escribe	lee	**Sí**
tUltimo	lee y escribe	—	**No**
total	—	lee y escribe	**No**

No importa dónde se escribe, sino cuántas partes del programa la tocan.

Por qué este error es traicionero

Hoy funciona

Con un Serial.println en el loop, el compilador suele volver a leer la RAM. Parece que volatile no hacía falta.

→

Mañana falla

Quitas el println o cambias la optimización, y el loop deja de ver la bandera.

No te fíes de "ya me funcionó": si cumple la regla, va volatile.

BLOQUE 8 · BANDERA

El buzón rural con banderita

Cartero = ISR

Deja la carta y levanta la banderita. Tarda 5 segundos y se va.

Banderita = variable

bool volatile alerta.

Arriba = "pasó algo".

Tú = loop()

Ves la banderita, la bajas y lees la carta con calma.

El cartero no entra a la casa a leerte la carta en voz alta.

División del trabajo

ISR (rápida)

1. Sube la bandera: alerta = true
2. Sale.

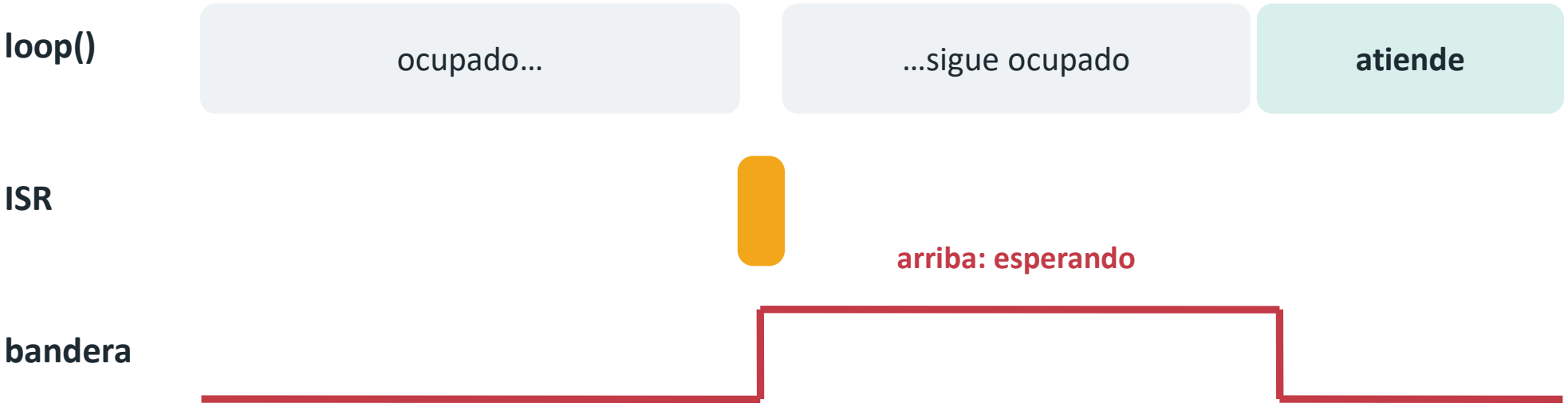
Nada más.

loop() (con calma)

1. ¿Bandera arriba?
2. La baja: alerta = false
3. Hace el trabajo pesado: imprimir, sirena, Wi-Fi, LCD…

En el loop: primero bajar la bandera, después trabajar.

La bandera en el tiempo



El aviso puede atenderse un poco tarde, pero no se pierde.

La bandera en código

```
volatile bool alerta = false;        // 1. la bandera

void IRAM_ATTR isrBoton() {          // 2. la ISR
  alerta = true;                     //    sube la bandera
}

void loop() {                        // 3. el loop
  if (alerta) {                      //    ¿arriba?
    alerta = false;                  //    la baja
    Serial.println("¡Alerta!");      //    trabajo pesado
  }
}
```

Limitación: la bandera no cuenta

Bandera (bool)

Dice "pasó algo".

3 toques mientras el loop está ocupado = 1 aviso.

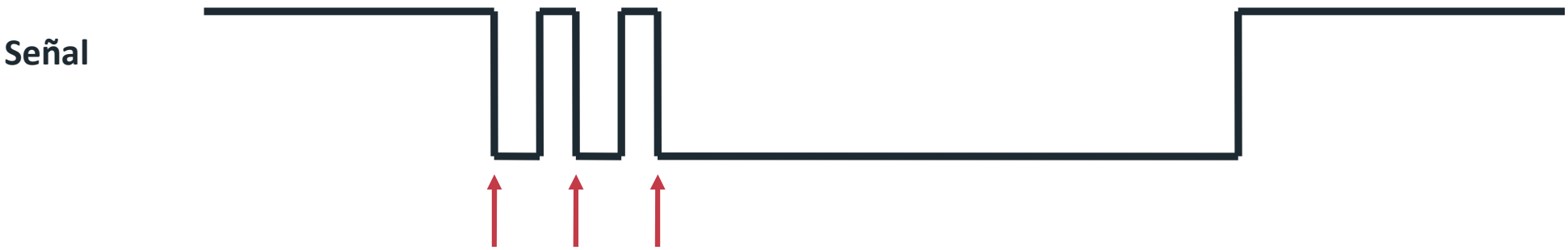
Contador (uint32_t)

Dice "cuántas veces".

La ISR suma 1; el loop lo lee.

```
volatile uint32_t pulsos = 0;
void IRAM_ATTR isrBoton() { pulsos++; }
```

El rebote: un toque, varios flancos



3 flancos de bajada en unos milisegundos → la ISR corre 3 veces

Con sondeo (AP2)

El loop() filtraba el rebote.

Con interrupción

Cada rebote dispara la ISR: un toque cuenta como 3 o 5.

Antirrebote dentro de la ISR, con millis()

```
uint32_t tUltimo = 0;  // solo ISR

void IRAM_ATTR isrBoton() {
  uint32_t ahora = millis();
  // ¿toque nuevo?
  if (ahora - tUltimo >= 50) {
    alerta  = true;
    tUltimo = ahora;
  }  // si no: es rebote
}
```

La idea

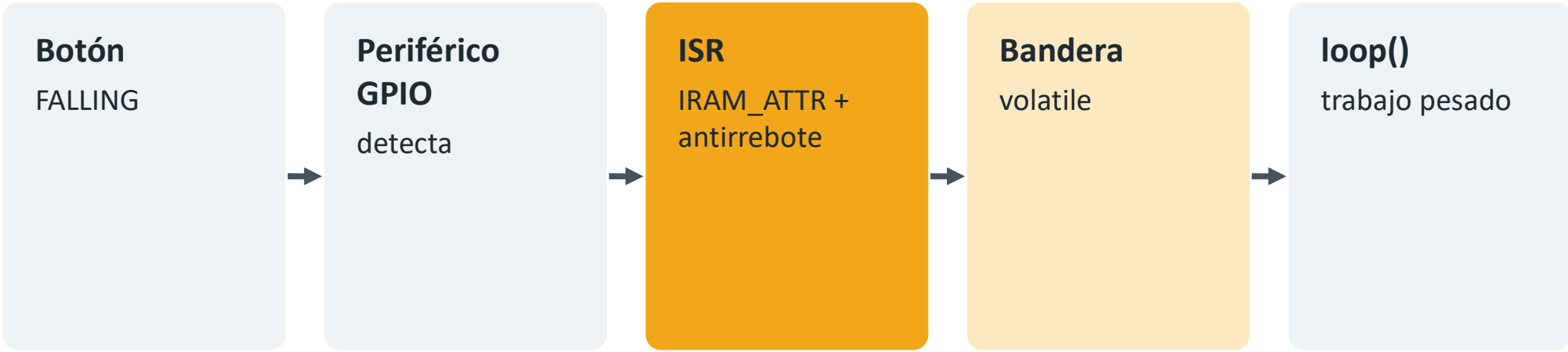
Ignorar flancos a menos de 50 ms del último aceptado.

Ya lo conocen

Es el patrón ahora − tAnterior de la semana 2.

BLOQUE 10 · TODO JUNTO

El recorrido completo de un aviso



¿Qué pieza falla si olvidas volatile? ¿Y si olvidas IRAM_ATTR?

Las piezas juntas (1/2): declaraciones e ISR

```
const uint8_t  PIN_BOTON = 21;
const uint8_t  PIN_LED   = 19;
const uint32_t T_REBOTE  = 50;       // ms

volatile bool alerta = false;        // ISR y loop
uint32_t tUltimo = 0;                // solo la ISR

void IRAM_ATTR isrBoton() {
  uint32_t ahora = millis();
  if (ahora - tUltimo >= T_REBOTE) {
    alerta  = true;
    tUltimo = ahora;
  }
}
```

Las piezas juntas (2/2): setup() y loop()

```
void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED, OUTPUT);
  pinMode(PIN_BOTON, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_BOTON),
                  isrBoton, FALLING);
}

void loop() {
  if (alerta) {
    alerta = false;
    digitalWrite(PIN_LED, !digitalRead(PIN_LED));
    Serial.println("¡Alerta! Puerta abierta");
  }
  delay(2000);   // loop ocupado: el aviso NO se pierde
}
```

Control de aprendizaje

5
preguntas

10
minutos

1
hoja por persona

Individual · sin apuntes ni celular · escribe tu nombre · justifica cada respuesta: el "por qué" vale más que la respuesta.

CONTROL · PREGUNTA 1 DE 5

Flanco correcto

1

Un pulsador está conectado con INPUT_PULLUP. La alarma debe activarse en el instante en que alguien presiona el botón.

¿Qué modo usas en attachInterrupt: RISING, FALLING o CHANGE? Justifica.

Responde en tu hoja y justifica: el "por qué" vale más que la respuesta.

CONTROL · PREGUNTA 2 DE 5

¿Cuáles necesitan volatile?

2

En un programa hay tres variables globales. ¿Cuáles deben declararse volatile? Justifica cada una.

```
uint32_t pulsos;   // la ISR suma 1; el loop lee
uint32_t tUltimo;  // solo la usa la ISR
uint32_t total;    // solo la usa el loop
```

Responde en tu hoja y justifica: el "por qué" vale más que la respuesta.

CONTROL · PREGUNTA 3 DE 5

El ESP32 se reinicia

3

Un estudiante escribe Serial.println("toque") dentro de la ISR. De vez en cuando el ESP32 se reinicia y el Monitor Serie muestra "Guru Meditation Error".

¿Qué está pasando y cómo lo corriges?

Responde en tu hoja y justifica: el "por qué" vale más que la respuesta.

CONTROL · PREGUNTA 4 DE 5

Sin antirrebote

4

Se quita el antirrebote de la ISR (T_REBOTE = 0) y se presiona el botón 10 veces.

¿Cuántos avisos esperas: exactamente 10, menos de 10 o más de 10? Justifica.

Responde en tu hoja y justifica: el "por qué" vale más que la respuesta.

CONTROL · PREGUNTA 5 DE 5

Repaso de temporizadores (semana 3)

5

Un temporizador cuenta a 1 MHz (una cuenta cada 1 µs). La alarma está en 500 000 con autorrecarga.

¿Cada cuánto tiempo se ejecuta la ISR? ¿Cuántas veces por segundo? Muestra el cálculo.

Responde en tu hoja y justifica: el "por qué" vale más que la respuesta.